



17. Python: Random Numbers

"It is actually better when randomness is involved"

Previous Section:

 [16. Python: Date and Time Functions](#)

Contents:

[1. Common Random Functions](#)

[2. Special Randomization](#)

[Summary](#)

[References](#)

Sometimes, asking users for input becomes repetitive. What if we want a game enemy to attack differently every time? Or generate random passwords, shuffle cards, simulate dice, or randomly select data?

That is where Python's built-in `random` module becomes useful. The `random` module allows Python to generate pseudo-random values — numbers or selections that appear random to us.

To use it:

```
import random
```

Or:

```
from random import *
```

Although technically possible like any other libraries, the second style is not commonly used because it when it comes to the random module, because it

***pollutes the local namespace** with many common function names. The module, as you can see, has really generic names like `seed()` and `sample()` that can easily overwrite your own variables and cause unexpected bugs. Furthermore, using standalone functions like `random()` reduces your code's readability because it becomes unclear whether the name refers to a local variable or an imported tool.*

Randomness is extremely important in: Games; Simulations; Machine Learning; Security systems; Data sampling; AI behavior; Scientific experiments

And interestingly enough, computers are not truly random. Most randomness in programming is actually generated mathematically using algorithms.

1. Common Random Functions

- `random.choice()` : Select a random element from a sequence. Works with lists, tuples, strings. But not sets and dictionaries (unless converted first)

```
import random

fruits = ["apple", "banana", "orange", "grape"]
print(random.choice(fruits))
```

Possible Output: banana

You can even use strings:

```
word = "PYTHON"
print(random.choice(word))
```

Possible Output: 1

- `random.choices()` : Select multiple random elements **with replacement**. This means the same element can appear multiple times.

```
import random

numbers = [1, 2, 3, 4]
print(random.choices(numbers, k=5))
```

Possible Output: [2, 4, 2, 1, 1]

Notice how duplicates can appear.

- `random.sample()` : Select multiple unique random items. Unlike `choices()`, duplicates are not allowed.

```
students = ["Alex", "John", "Emma", "Sophia"]
print(random.sample(students, 2))
```

Possible Output: ['Emma', 'John']

- `random.randint()` : Generate a random integer between two values (inclusive).

```
print(random.randint(1, 10))
```

Possible Output: 7

The number can be from 1 to 10.

- `random.random()` : Generate a random floating-point number between 0 and 1.

```
print(random.random())
```

Possible Output: 0.4839201

Range: $0 \leq x < 1$

- `random.uniform()` : Generate a random float between two numbers.

```
print(random.uniform(1, 5))
```

Possible Output: 3.72819

- `random.randrange()` : Works similarly to `range()` but randomly selects a value.

```
print(random.randrange(0, 20, 2))
```

Possible Output: 14

This selects a random even number from: 0, 2, 4, 6, ... up to, but not including 20.

- `random.shuffle()` : Shuffle a mutable sequence like a list.

```
cards = ["A", "K", "Q", "J"]  
random.shuffle(cards)  
  
print(cards)
```

Possible Output: ['Q', 'A', 'J', 'K']

This modifies the original list directly.

- `random.seed()` : Set a starting point for randomness. This creates **constant randomness**. Meaning the same seed value will generate the same sequence of random numbers every time.

```
random.seed(42)  
print(random.randint(1, 100))  
print(random.randint(1, 100))
```

Output (always the same):

82

15

The number 42 itself is not special. Any number can be used as a seed. This is useful for testing, debugging, machine Learning experiments, reproducible simulations.

2. Special Randomization

Randomness is not limited to simple numbers. You can combine formulas and mathematics with random values to create dynamic systems.

- For example:

```
damage = random.randint(10, 20) * 1.5
print(damage)
```

Possible Output: 24.0

- You can also simulate dice:

```
dice = random.randint(1, 6)
print(dice)
```

Possible Output: 2

- Or create random passwords:

```
characters = "abcdefghijklmnopqrstuvwxyz123456789"

password = "".join(random.choices(characters, k=8))
print(password)
```

Possible Output: x7k2pq1z

You can even randomize formulas:

```
x = random.uniform(1, 10)

result = x ** 2 + 5
print(result)
```

Possible Output: 24.164454357946596

Here, the formula changes every time because `x` is random.

Randomness and Machine Learning: Interestingly, randomness is heavily used in Machine Learning. For example:

- Random initialization helps Neural Networks learn
- Random data splitting prevents overfitting
- Random Forest models use randomness internally
- Stochastic algorithms depend on random sampling

Without randomness, many Machine Learning systems would become biased or repetitive.

Summary

This section introduced Python's `random` module and some of its most important functions.

We learned how to:

- Select random values
- Generate random integers and floats
- Shuffle lists
- Sample unique items
- Create reproducible randomness using seeds

The purpose of randomness is exactly what the name suggests: To introduce unpredictability into programs. And while randomness may seem simple at first, it becomes one of the most important ideas in: Games; Simulations; Artificial Intelligence; Security; Machine Learning; Scientific computing.

Because sometimes, the best systems are the ones that do not behave exactly the same every time.

References

<https://docs.python.org/3/library/random.html>

<https://www.geeksforgeeks.org/python/python-random-module/>

Next Section:

[`</>` 18. Python: Lambda Functions](#)